Comparativa entre tipos de bases de datos, análisis de principales caracteristicas y ventajas de uso.

Sebastian Joel Rivett Macias
INAOE - Estudiante de MCC
sebastian.rivett@inaoe.mx

***Abstract*—Se abordan los tipos de bases de datos: Distribuidas, NoSQL, Serverless, Tolerantes a Fallos, con Chekpointing , y basadas en el framework DELTA LAKE, considerando los conceptos principales para entender de manera básica su funcionamiento, cuales son las principales caracteristicas que las componen y dentro de que ambientes serian ideal utilizar cada una y el porque, así como cuales son las tendencias actuales. Se indago sobre su correspondiente funcionamiento sobre las siguietnes bases de datos: PouchDB, MongoDB, Amazon DynamoDB, Apache Couchbase, CockroachDB Cloud, Amazon Aurora Serverless, RIAK, MYSQL Cluster, SQLITE, NEO4J, considerando cada uno de los tipos descritos en el artículo, esto para entender que caracteristicas comparten en común y cuales podrian ser sus casos de uso.**

***Index Terms*—Distribuidas, NoSQL, Serverless, Tolerantes a Fallos, Chekpointing, PouchDB, MongoDB, Amazon DynamoDB, Apache Couchbase, CockroachDB Cloud, Amazon Aurora Serverless, RIAK, MYSQL Cluster, SQLITE, NEO4J, DELTA LAKE.**

I. Introducción

A continuación se abordan los conceptos más necesarios en cada uno de los tipos de bases de datos, así como conceptos adicionales para entender algunas bases de datos ya existentes.

A. *Bases de Datos Distribuidas*

Sistemas que almacenan datos en múltiples ubicaciones físicas, pero se comportan como un único sistema unificado, permitiendo un acceso transparente y simultáneo desde diferentes sitios [1] [2] [3]. La arquitectura de las bases de datos distribuidas está diseñada para optimizar el almacenamiento, la recuperación y el procesamiento de datos distribuidos geográficamente en múltiples sitios. Esta configuración posee varias características clave que facilitan la gestión eficiente de datos en entornos complejos y dispersos.

- **Transparencia de Localización:** Ocultar la ubicación física de los datos. Se refiere a la capacidad de acceder a los datos sin necesidad de saber dónde están físicamente almacenados. Es como pedir una pizza usando una app; no es necesario saber de qué sucursal vendrá, solo esperar a que llegue la pizza.
- **Independencia de Fragmentación:** Manejar cómo se dividen y almacenan los datos. La fragmentación puede ser horizontal (dividir una tabla por filas) o vertical (dividir una tabla por columnas).
- **Autonomía Local:** Permite que cada nodo de la base de datos opere de manera independiente, gestionando sus propios datos sin necesidad de intervención constante de un nodo central. Esto es similar a cómo diferentes oficinas de una empresa pueden operar localmente, pero aún forman parte de la organización más grande.
- **Sincronización:** Mantener los datos consistentes a través de la red. Asegurando que todos los cambios realizados en una ubicación se reflejen en todas las demás. Cuando se edita un documento en Google Docs; los cambios que haces son vistos en tiempo real por todos los usuarios que tienen acceso al documento.
- **Manejo de Fallos:** Estrategias para recuperarse de errores en nodos específicos. Esto puede ser comparado con un equipo de fútbol: si un jugador se lesiona, otro jugador puede tomar su lugar para asegurar que el equipo siga funcionando eficientemente.
- **Seguridad:** Proteger los datos en un entorno distribuido. Esto incluye medidas como cifrado, autenticación y control de acceso para asegurar que solo los usuarios autorizados puedan acceder a los datos. Se puede comparar al sistema de seguridad en una casa, donde diferentes puertas pueden tener diferentes cerraduras y alarmas.
- **Escalabilidad:** Aumentar capacidad al añadir más nodos. Es posible mejorar el rendimiento y la capacidad de almacenamiento de la base de datos simplemente por añadir más servidores. Esto es similar a añadir más vagones a un tren para aumentar la capacidad de pasajeros.
- **Balanceo de Carga:** Distribuir operaciones uniformemente entre nodos. El objetivo es maximizar la eficiencia evitando que algunos nodos estén sobrecargados mientras otros están subutilizados. Se puede ver como tener varias cajas registradoras en un supermercado: si una fila es demasiado larga, los clientes pueden ir a otra caja para ser atendidos más rápidamente, distribuyendo así la carga de trabajo de manera más equitativa entre todas las cajas.

Algunas bases de datos distribuidas son: **Apache Cassandra , Apache Hbase , Apache CouchDB , PouchDB , MongoDB , Riak , Amazon DynamoDB , Redis , CockroachDB**

B. *Sincronización en Bases de Datos Distribuidas*

1) Mecanismos de Consistencia de Datos: : Existen diferentes mecanismos para mantener la coherencia y consistencia de los datos a traves de los nodos de una base de datos

distribuida, esto se logra manteniendo diferentes niveles de consistencia en los datos. [3] [54]

- **Consistencia Fuerte:** Cada operación de lectura recupera el valor más reciente escrito en la base de datos. Esto asegura que todas las copias de un dato en la red están sincronizadas inmediatamente después de una actualización.
- **Consistencia Debil:** Las actualizaciones pueden tardar algún tiempo en propagarse a todos los nodos, lo que puede resultar en lecturas que no recuperan el valor más reciente pero eventualmente alcanzan la consistencia después de que no se realizan más actualizaciones.
- **Consistencia Eventual:** Garantiza que, si no se realizan nuevas actualizaciones, todas las réplicas en la red eventualmente convergerán al último valor actualizado, pero no especifica cuándo ocurrirá esto.

2) Protocolos de Sincronización : Para asegurar la sincronización es necesario considerar cierto tipo de dato o bandera que así lo permita, por lo que se tienen diferentes maneras de abordar este problema: [3] [54]

- **Algoritmos de bloqueo distribuido:** Se utilizan para controlar el acceso a los datos que están siendo leídos o modificados por una transacción, asegurando que no se produzcan conflictos entre transacciones concurrentes.
- **Timestamps:** Cada transacción recibe una marca de tiempo única que ayuda a ordenar las transacciones de forma consistente en todos los nodos. Esto se utiliza para resolver conflictos y garantizar que las operaciones se realicen en el orden correcto.
- **Two-phase commit (2PC):** Es un protocolo que asegura que todas las partes (nodos) involucradas en una transacción estén de acuerdo antes de comprometer una transacción. Si algún nodo falla o no está de acuerdo, la transacción se aborta en todos los nodos.

3) Desafíos de la Sincronización : Se presentan diferentes afectaciones a la sincronización, estas mismas deben de ser contempladas para implementarse en una base de datos que sea utilizada en un entorno ”real”. [3] [54]

- **Latencia de red - Velocidad de Canal Comunicación:** Que tan rapido se conectan los nodos y cuanto retraso se puede tener a la hora de realizar una petición.
- **Particiones de red:** Como se puede llegar a segmentar el sistema distribuido debido a fallos presentes en la red, haciendo momentaneamente imposible la sincronización.

4) Aplicaciones Prácticas y Teoría Relacionada : Para mitigar los desafios presentes en la sincronización se debe de considerar alguno de los siguientes modelos. [3] [54] [55]

- **Teorema CAP:** Sirve para entender las limitaciones inherentes a cualquier sistema distribuido en términos de consistencia, disponibilidad y tolerancia a particiones.
- **Modelos de Coherencia:** Diferentes modelos de coherencia, como coherencia de lectura, coherencia de escritura, y coherencia causal, ofrecen diversos grados y estrategias de sincronización, adaptándose a diferentes tipos de aplicaciones y requisitos operativos.

C. Bases de Datos NoSQL

Sistemas de almacenamiento de información que no utilizan el modelo relacional típico de SQL. Son útiles para manejar grandes volúmenes de datos dispersos y estructuras de datos variadas [4] [5] [6]. Para entender en mejor medida este tipo de base de datos se recomienda considerar los siguientes conceptos / características principales:

- **Modelos de Datos:** Clave-valor, documentos, columnares, y grafos, así como Unique ID (FECHA - HORA).
- **Escalabilidad Horizontal:** Añadir más servidores fácilmente para manejar más datos.
- **Consistencia Eventual:** Asegurar que los datos eventualmente sean consistentes en todos los nodos.
- **Soporte para Big Data:** Manejo eficiente de grandes volúmenes de datos.
- **Flexibilidad de Esquemas:** Adaptarse a cambios en los modelos de datos sin downtime.
- **Rendimiento:** Optimizado para operaciones rápidas de lectura y escritura.
- **Resistencia a Particiones:** Funcionar a pesar de fallos de red parciales.

Algunas bases de datos NoSQL son: **Apache Cassandra , Apache Hbase , Apache CouchDB , PouchDB , MongoDB , Riak , Amazon DynamoDB , Redis , Elasticsearch , Apache Couchbase**

D. Bases de Datos Serverless

Servicios de bases de datos que eliminan la necesidad de administrar la infraestructura. El proveedor de la nube maneja automáticamente el escalado y la optimización, y los usuarios pagan solo por los recursos que utilizan. [9] [11] [12] [13]. Para entender en mejor medida este tipo de base de datos se recomienda considerar los siguientes conceptos / características principales:

- **Elasticidad Automática:** Ajustar recursos automáticamente dependiendo de la carga.
- **Modelo de Pago por Uso:** Costos basados en el consumo exacto de recursos.
- **Gestión Simplificada:** El proveedor maneja la infraestructura y el mantenimiento.
- **Integración con la Nube:** Conectar fácilmente con otros servicios en la nube.
- **Desarrollo Ágil:** Permite a los desarrolladores enfocarse en el código sin preocuparse por la administración de la base de datos.

Algunas bases de datos Serverless son: **Amazon Aurora Serverless , PlanetScale , CockroachDB , MongoDB Atlas , FaunaDB , Firebase Realtime Database**

E. Bases de Datos Tolerantes a Fallos

Sistemas diseñados para garantizar la disponibilidad y la integridad de los datos incluso cuando ocurren fallos

en componentes del sistema, mediante técnicas como replicación y recuperación automática. [14] [15] [16] [56] Para entender en mejor medida este tipo de base de datos se recomienda considerar los siguientes conceptos / características principales:

- **Replicación:** Copias de datos para permitir la continuidad del servicio. Esta característica implica mantener copias de los datos en múltiples ubicaciones o nodos. Al hacerlo, si uno de los nodos falla, otro puede tomar su lugar sin interrumpir el servicio. Puede ser visto al equivalente de tener varios discos de música guardados en diferentes lugares. Si uno se daña, se puede seguir disfrutando de la música desde otro disco sin problemas.
- **Recuperación Automática:** Consiste en la capacidad del sistema para reanudar las operaciones automáticamente después de un fallo. Cuando una persona está viendo una película en streaming y se pierde la conexión, el sistema automáticamente busca otra red disponible. Esto permite que el espectador pueda continuar disfrutando de la película sin intervención manual.
- **Failover Automático:** Cambiar a sistemas de respaldo sin intervención manual. En caso de un fallo, este mecanismo permite que la base de datos cambie automáticamente a un sistema de respaldo. Puede ser visto al equivalent de contar con un generador de emergencia en un hospital que se activa automáticamente si se corta la electricidad, asegurando que los equipos esenciales sigan funcionando.
- **Balanceo de Carga:** Distribuir cargas para reducir puntos de fallo. Esto puede compararse a una autopista con múltiples carriles; si un carril está bloqueado o congestionado, los vehículos pueden desviarse a otros carriles, manteniendo el flujo del tráfico.
- **Monitoreo Continuo:** Supervisión constante para detectar y resolver problemas rápidamente. El parecido al sistema de alarma presente en una casa, cuando suena dicha alarma se sabe que algo esta mal o puede ir mal (alarma de temblores, incendio, tornado, huracan etc).

Algunas bases de datos Tolerantes a Fallos son: **MySQL Cluster , Apache Cassandra , Apache CouchDB , PouchDB , MongoDB , Riak , Amazon DynamoDB , Redis**

F. *Replicación en Bases de Datos Tolerantes a Fallos*

1) Mecanismos de Replicación :

- **Replicación Síncrona:** Las actualizaciones a un nodo se propagan inmediatamente a todas las réplicas, y la transacción no se considera completa hasta que todas las réplicas han sido actualizadas. Este enfoque garantiza una alta coherencia de datos, pero puede reducir el rendimiento debido a la latencia en la confirmación de todas las réplicas. [56]
- **Replicación Asíncrona:** Las actualizaciones a las réplicas se realizan de manera diferida, lo que permite una mayor velocidad de operación local en el nodo principal. Sin embargo, esto puede llevar a inconsistencias temporales entre nodos, lo cual es aceptable en escenarios donde se puede tolerar una coherencia eventual. [56]

2) Teoría de Consistencia :

- **Consistencia Estricta:** Cualquier lectura de un dato refleja el valor más reciente escrito en él. Es el modelo más restrictivo y difícil de mantener en ambientes distribuidos debido a las demandas de sincronización y latencia.
- **Consistencia Causal:** Permite que las actualizaciones que no están causalmente relacionadas puedan ser vistas en un orden diferente por diferentes nodos. Este modelo es menos restrictivo que la consistencia estricta y es suficiente para muchas aplicaciones prácticas. [56]
- **Consistencia Eventual:** Todas las copias convergen al mismo valor si no se realizan más actualizaciones. Este modelo es común en sistemas donde la disponibilidad y el rendimiento son críticos, y se puede tolerar una inconsistencia temporal. [56]

G. *Balanceo de Carga en Bases de Datos Tolerantes a Fallos*

1) Algoritmos de Balanceo de Carga :

- **Round Robin:** Distribuye las solicitudes de manera secuencial y equitativa entre todos los nodos. Es simple y efectivo en muchos escenarios, aunque no considera la carga actual de cada nodo. [56]
- **Least Connections:** Dirige nuevas solicitudes al nodo con el menor número de conexiones activas, lo que puede ser más justo en situaciones donde las solicitudes varían significativamente en términos de carga de procesamiento. [56]
- **Hash Consistente:** Utiliza una función hash para distribuir las solicitudes basadas en ciertas claves, lo que es útil para mantener la localidad de referencia y reducir la cantidad de redistribuciones de carga en el caso de que se añadan o se retiren nodos. [56]

2) Teoría de Colas : Modela los sistemas distribuidos como redes de colas, donde las solicitudes son procesadas por servidores. Los modelos matemáticos ayudan a predecir tiempos de respuesta y a optimizar la configuración de los recursos según los patrones de llegada y servicio de las solicitudes. [56]

H. *Bases de Datos con Checkpointing*

Bases de datos que regularmente guardan una instantánea de su estado para facilitar una rápida recuperación en caso de fallos, reduciendo el tiempo necesario para restaurar el sistema después de un problema. [17] [18] [19] [20] Para entender en mejor medida este tipo de base de datos se recomienda considerar los siguientes conceptos / características principales:

- **Puntos de Control:** Guardar el estado de la base de datos en intervalos regulares para facilitar la recuperación. Se puede pensar en esto como hacer copias de seguridad de un documento mientras se trabaja en él, para no perder todo el progreso si la computadora se apaga repentinamente.

- **Recuperación Rápida:** Utilizar puntos de control permite restaurar el sistema más rápidamente después de un fallo. En lugar de reconstruir la base de datos desde cero, el sistema puede reanudarse desde el último estado guardado, acelerando significativamente el proceso de recuperación.
- **Minimización de la Pérdida de Datos:** Reducir el riesgo de pérdida de datos mediante checkpoints frecuentes. Esto es como guardar constantemente un juego en diferentes puntos; si el jugador pierde, puede volver al último punto guardado en lugar de empezar de nuevo desde el principio.
- **Eficiencia en el Rendimiento:** Balances entre la frecuencia de checkpoints y el rendimiento general del sistema. Es similar a decidir cuántas veces se debe guardar un proyecto: demasiado a menudo puede ser disruptivo, y hacerlo raramente puede ser riesgoso.
- **Automatización:** Configuración de checkpoints automáticos basados en tiempo o eventos. Se puede comparar con un sistema de riego automático que se activa en momentos específicos para asegurar el cuidado adecuado de las plantas.

Algunas bases de datos con Checkpointing son: **PostgreSQL , MySQL/InnoDB , Oracle Database , SQLite , DB2 , Redis , Neo4j , MongoDB**

1. Puntos de Control en Bases de Datos con Checkpointing

Para considerar los puntos de control en bases de datos con checkpointing, es necesario considerar los modelos de consistencia previamente descritos en secciones anteriores, Mecanismos de Chekpointing y como se realiza, y la Recuperación basada en Logs. [56]

1) Mecanismos de Checkpointing : Los mecanismos implicados incluyen:

- **Checkpoint Síncrono:** Todas las consultas se pausan mientras se escribe el estado actual al almacenamiento persistente. Esto garantiza que no haya inconsistencias en el punto de control, pero puede introducir latencia en el sistema durante el proceso.
- **Checkpoint Asíncrono:** Las consultas continúan mientras se realiza el checkpoint. Este método reduce el impacto en el rendimiento del sistema pero puede complicar el proceso de recuperación, ya que se deben considerar las consultas que se ejecutaron durante la toma del checkpoint.

2) Teoría de Recuperación Basada en Log : La teoría de recuperación basada en log utiliza un log de transacciones para registrar los cambios en la base de datos antes de que se apliquen realmente a la misma. En caso de fallo, el sistema puede usar este log para:

- **Redo:** Rehacer las operaciones que se habían completado antes del fallo pero que no se habían persistido completamente en la base de datos.
- **Undo:** Deshacer las operaciones que estaban en proceso en el momento del fallo y que podrían haber dejado el sistema en un estado inconsistente.

II. Metodología de Funcionamiento de las Bases de Datos

A continuación se menciona el funcionamiento principal de cada una de las bases de datos consideradas, cabe destacar que se busco describir de la maera más simple cada una de los tipos de bases de datos selecionadas.

A. Base de Datos MongoDB

Para crear una base de datos MongoDB, se debe de considerar cualqueira de los siguientes entornos: **MongoDB Atlas**, un servicio completamente gestionado para despliegues en la nube; **MongoDB Enterprise**, la versión de suscripción autogestionada; y **MongoDB Community**, la versión gratuita y autogestionada. [32] [8]

Un registro en MongoDB es un documento, una estructura de datos de pares campo-valor similar a objetos JSON. Los valores pueden incluir documentos, arreglos y arreglos de documentos. Los documentos facilitan la programación al corresponder a tipos de datos nativos y reducir la necesidad de uniones costosas gracias a esquemas dinámicos y polimorfismo fluido. MongoDB almacena documentos en colecciones, similares a las tablas en bases de datos relacionales, y soporta vistas de solo lectura y vistas materializadas bajo demanda. [32] [8]

Fig. 1. Estructura de registros presente en MongoDB. [32]

Organiza los datos en colecciones y documentos, donde cada documento puede tener un esquema diferente.

Soporta índices secundarios, lo que permite consultas eficientes y la indexación de cualquier campo de un documento. Los datos se almacenan en un formato binario llamado BSON (Binary JSON), optimizado para ser ligero, traversable y eficiente.

B. Base de Datos RIAK

Riak es una base de datos NoSQL distribuida y de código abierto optimizada para entornos en la nube que ofrece alta escalabilidad y tolerancia a fallos. No depende de un servidor maestro, lo que elimina puntos únicos de fallo y permite una disponibilidad continua incluso cuando algunos nodos fallan. Riak es especialmente adecuada para aplicaciones web que requieren alta disponibilidad y escalabilidad, permitiendo añadir nodos sin interrupciones y reduciendo el costo operativo gracias a su diseño sencillo. [39] [40]

Ademas de ello no cuenta con un nodo maestro, por lo que no tiene ningún punto único de anomalía. Es Tolerante a errores debido a su naturaleza distribuida, pudiendo perder el acceso a nodos de Riak, pero no perderá nunca los datos. Convierte a

los datos en altamente disponibles para su uso en aplicaciones web de lectura y escritura intensivas. Escala de forma sencilla e inteligente, permitiendo añadir fácilmente un nodo a un clúster, sin tiempo de inactividad. [39]
Usa un esquema clave-valor sin esquema fijo, lo que permite una gran flexibilidad en los tipos de datos almacenados. La distribución de los datos se gestiona a través de un anillo consistente que reparte las claves uniformemente entre los nodos del clúster. [39] [40] [41]
Riak utiliza el hash consistente para mapear objetos a lo largo del borde de un círculo (el anillo). Las particiones en Riak no se organizan alfabéticamente; en cambio, una partición indica un rango de hashes de clave (función SHA-1 aplicada a una clave). [57]

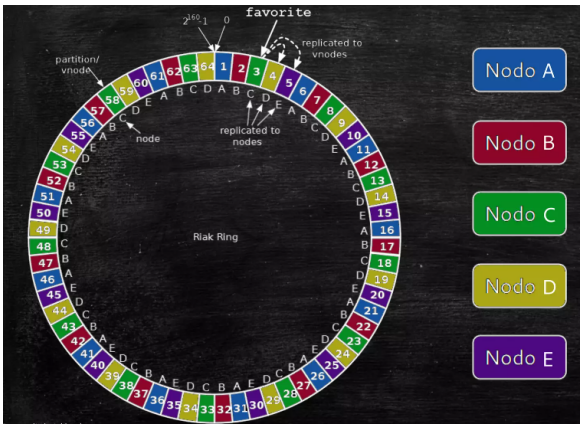

Fig. 2. Riak Ring: Esquema de Replicación resistente a fallos de Riak [41]

Para considerar la arquitectura de RIAK, es necesario tomar en cuenta cada una de las partes en las que se subdivide:

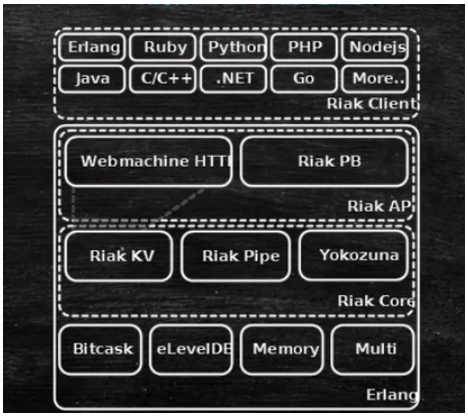

Fig. 3. Arquitectura de RIAK [59]

- **Erlang:** Cuando se inicia un nodo Riak, también se activa una máquina virtual de Erlang (VM) para ejecutar y gestionar los procesos de Riak. Esto incluye vnodes, mensajes de proceso, rumores, gestión de recursos y más. El proceso del sistema operativo Erlang se ejecuta a través del comando beam.smp con muchos argumentos, que se configuran a través del archivo etc/riak.conf. [59]
- **RIAK Core:** Comparte responsabilidades con los proyectos que se construyen sobre él para gestionar el espacio de claves particionado, lanzar y supervisar vnodes, construir listas de preferencias, realizar transferencias de datos sugeridas y otras funciones que no están directamente relacionadas con las interfaces de cliente, el manejo de solicitudes o el almacenamiento. [59]
- **RIAK API:** Representa el aspecto de la interfaz con el cliente de Riak. Las implementaciones manejan cómo se codifican y transfieren los datos, y este proyecto maneja los servicios para presentar esas interfaces, gestionar conexiones y proporcionar puntos de entrada. [59]

C. *Base de Datos NEO4J*

Base de datos orientada a grafos, desarrollada por Neo Technology Inc e implementada en Java, que almacena datos estructurados como grafos dirigidos en lugar de tablas, facilitando la representación de relaciones complejas entre datos. Disponible en Windows, Linux y OS X, Neo4j ofrece tanto una versión gratuita como una de pago, esta última con características adicionales como replicación y alta disponibilidad. La base de datos soporta múltiples lenguajes de consulta, destacando Cypher para la manipulación de grafos y Gremlin para su gestión. Los usuarios pueden acceder a los datos a través de una consola de texto, un entorno web gráfico y APIs. Neo4j es reconocida como la base de datos de grafos más popular y utiliza un modelo de grafos de propiedades etiquetados. [47] [48] [49]
Las consultas se hacen en el servidor haciendo uso del algoritmo de búsqueda por profundidad (BFS) a partir del nodo seleccionado realizando consultas a Neo4j para cada tipo de relación donde una de sus partes sea el nodo actual, hasta llegar al nivel de profundidad solicitado. Se van guardando los nodos y los arcos para devolverlos como resultado. [47] [48] [49]
Los nodos y relaciones están indexados para acelerar las consultas de grafo, permitiendo consultas eficientes sobre relaciones complejas.
Utiliza nodos, relaciones, propiedades y etiquetas para representar y almacenar datos en forma de grafo. [47] [48] [49]

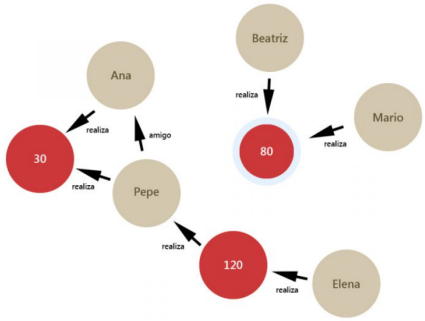

Fig. 4. Representación en Grafos de la estructura columnar de las tablas. [48]

D. *Base de Datos PostgreSQL*

Sistema de gestión de bases de datos relacional (RDBMS) de código abierto que destaca por su robustez y conformidad con los estándares SQL. Es conocido por su arquitectura avanzada y soporte para características modernas como consultas complejas y transacciones ACID (Atomicity, Consistency, Isolation, Durability).
Utiliza un modelo de datos relacional donde la información se almacena en tablas compuestas por filas y columnas.
Soporta un amplio rango de tipos de datos primitivos, y permite la creación de tipos definidos por el usuario.
Incorpora un sofisticado sistema de control de concurrencia multiversión (MVCC) que minimiza los bloqueos sin comprometer la integridad.
Los datos se almacenan en bloques dentro de archivos en el sistema de archivos del servidor, lo que facilita operaciones eficientes de lectura y escritura.
Implementa índices, incluidos B-tree, hash, GiST, SP-GiST, BRIN y GIN, que ayudan a mejorar el rendimiento de las consultas. [57]

```
SELECT city, temp_lo, temp_hi, prcp, date FROM weather;
```

The output should be:

```
     city      | temp_lo | temp_hi | prcp |    date
---------------+---------+---------+------+------------
 San Francisco |      46 |      50 | 0.25 | 1994-11-27
 San Francisco |      43 |      57 |    0 | 1994-11-29
 Hayward       |      37 |      54 |      | 1994-11-29
(3 rows)
```

Fig. 5. Consultas realizadas en entorno con POSTGRESQL [57]

E. *Bases de Datos basada en DATA LAKE*

Data Lake es un término que describe un sistema de almacenamiento masivo que permite almacenar grandes cantidades de datos crudos en su formato nativo. No está estructurado como una base de datos tradicional; más bien se trata de un repositorio que puede contener una amplia variedad de tipos de datos, desde archivos de texto hasta videos, pasando por datos de sensores y logs.
Los data lakes suelen estar basados en tecnologías de almacenamiento distribuido como Hadoop o sistemas basados en nube. Los datos en un data lake no necesitan ser preprocesados o transformados antes de su almacenamiento; se guardan en su forma origina, pudiendo lescalar a petabytes de información almacenada.
La organización dentro de un data lake es flexible, aunque se recomienda mantener algún nivel de estructuración para evitar el problema conocido como ”data swamp”, donde los datos se vuelven inaccesibles o inútiles debido a la falta de organización y metadatos. (Manejo jerarquico de fechas, o tags de captura presentes en la estructura de directorios).

F. *Base de Datos basada en DELTA LAKE*

Delta Lake es un marco de almacenamiento de datos de código abierto creado por Databricks en 2019 para mejorar la confiabilidad y el rendimiento de los data lakes, solucionando problemas como la consistencia y la calidad de los datos y la falta de transaccionalidad. Su diseño busca apoyar grandes cantidades de datos estructurados y no estructurados en data lakes, ofreciendo características como garantías ACID y mutabilidad de datos para facilitar el análisis y manipulación de datos masivos. Delta Lake permite operaciones complejas de análisis y aprendizaje automático en datos brutos, abordando las limitaciones de los data lakes tradicionales y evolucionando hacia plataformas unificadas de almacenamiento y análisis de datos. [53]

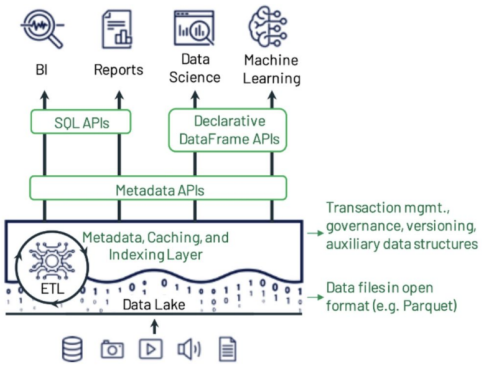

Fig. 6. Esquema de funcionamiento y uso de un Datalake [53]

Mientras que un data lake almacena grandes volúmenes de datos crudos en varios formatos, un data warehouse organiza los datos estructurados para optimizar el análisis y la generación de informes, pero puede ser limitante debido a su software propietario y la dificultad para manejar datos no estructurados. Por otro lado, un data lakehouse busca combinar las ventajas de ambos, ofreciendo la flexibilidad de un data lake con la estructura y gestión de un data warehouse en una única plataforma abierta y escalable. Delta Lake ayuda a preservar la integridad de los datos sin sacrificar el rendimiento, esencial para aplicaciones de análisis en tiempo real, inteligencia artificial y aprendizaje automático.

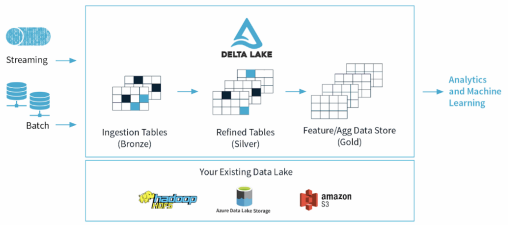

Fig. 7. Esquema de funcionamiento y uso de un Delta Lake basado en etapas de tablas. [53]

Facilita el seguimiento de los cambios en los datos y garantizan que estos sean consistentes y precisos. Esto se logra considerando un control de versiones de datos (time travel) el cual es bastante util para detectar cambios y

no afectar de manera directa el funcionamiento de tablas presentes en las bases de datos. Su principal archivo de almacenamiento suele ser formato parquet o DELTA [22] [23] [26] [27] [28] [29]. Cuenta con las siguientes características:

- **Formato de Almacenamiento Abierto:** Almacenar datos en un formato parquet optimizado para grandes datasets.
- **ACID Transactions:** Transacciones atómicas, consistentes, aisladas y duraderas para garantizar la integridad de los datos.
- **Time Travel (Data Versioning):** Ver y restaurar datos a un punto anterior en el tiempo.
- **Escalabilidad y Rendimiento:** Integrado con Apache Spark para procesamiento rápido.
- **Procesamiento unificado por lotes y en streaming:** Estrategias para recuperarse de errores en nodos específicos.
- **Aplicación y evolución de esquemas:** Administra y aplica automáticamente el esquema del lago de datos para evitar errores y permitir la evolución del esquema para facilitar la gestión de datos.
- **Técnicas de optimización:** Emplea técnicas de optimización avanzadas, como la omisión de datos y el orden z, para mejorar el rendimiento de las consultas.
- **Interoperabilidad con ecosistemas de datos:** Integración con otros componentes de los ecosistemas de datos y de IA para un análisis y procesamiento integral de datos.

G. *Bases de Datos basada en DELTA TABLES*

Delta Tables son estructuras específicas utilizadas dentro del entorno de Delta Lake para gestionar y operar sobre los datos almacenados.
Se trata de tablas presentes en la base de datos que aprovechan el formato parquet para almacenar datos de manera eficiente y que están optimizadas para operaciones de lectura y escritura rápidas.
Delta Tables soportan esquemas de datos que se pueden evolucionar con el tiempo sin necesidad de reescribir datasets enteros.
Los datos se almacenan en bloques distribuidos y replicados a través del sistema de archivos subyacente para mejorar la durabilidad y el acceso paralelo. [29]

III. Discusión y Conclusión

Los gestores de bases de datos son muy variados, tanto los presentes en el artículo como los presentes en las referencias, cada gestor tiene sus pros y contra para hacer uso de ellos en determinados entornos, no existe de momento un gestor de bases ded atos universal que considere las mejores características de cada uno de ellos ya que dependiendo del tipo de característica puede ser excluyente o no, o puede ser una solución demasiado robusta para el problema que se busca enfrentar. Centrandose en la parte de bases de datos distribuidas, tolerantes a fallos y con checkpointing se observo que actualmente existen diferentes esquemas para abordar cada uno de estos desafios dando así a lugar a diferentes gestores de bases de datos.
Si lo que se busca es almacenar los datos en esquema de tipo base de datos unicamente con considerar cualquier entorno simple de bases de datos sera suficiente (SQLlite, MYSQL etc), pero si se busca por ejemplo utilizar un entorno que cuente con un control de versiones y cambios de archivos bastante robusto se deberia de optar por utilizar el esquema de Delta Lake. Si lo que nos interesa es la replicación de archivos para obtener un sistema tolerante a fallos seria ideal entonces considerar RIAK, y si nos queremos centrar en realizar consultas y posteriores planes de debug una opción a considerar seria NEO4J por el esquema de trabajo de cada tabla.
No existe una comparativa ”justa” ya que depende del gestor de bases de datos elegido se puede hacer uso de diferentes caracteristicas y aprovechar estas mismas en las consultas a realizar para obterner miniaturas o datos de las tablas almacenadas, existen diferentes rubros en los cuales se puede comparar cada gestor de base de datos pero para que sea una comparativa justa se deberian de realizar en diferentes entornos ya que en algunos casos podria influir el hardware, software o sistema operativo de forma directa en el comportamiento asociado al gestor de la base de datos, asi como el ancho de banda de una red si se busca considerar comparar la parte de replicación así como el comportamiento del sistema distribuido presente.

A. *¿Porque MongoDB es más rapido que MYSQL?*

A continación se abodan las siguiente sideas las cuales permiten considerar una perspectiva más detallada del porque MongoDB es más eficiente en algunas tareas que MYSQL.
Arquitectura: MongoDB puede ser más adecuado para aplicaciones que requieren esquemas flexibles y escalabilidad global, lo que es común en aplicaciones modernas que manejan grandes volúmenes de datos variados y cambiantes. MySQL, por otro lado, podría ser preferible en entornos donde el rendimiento y la eficiencia en operaciones estructuradas son críticas, gracias a su optimización de almacenamiento y soporte multithreading. [58]
Almacenamiento y Flexibilidad: La flexibilidad de MongoDB en la gestión de datos lo hace superior para aplicaciones que necesitan adaptarse rápidamente a los cambios en los requisitos de datos o donde la estructura de los datos puede variar. Esto es especialmente útil en entornos donde los datos no están bien definidos o pueden cambiar con el tiempo. [58]
Rendimiento y Velocidad: MongoDB muestra un mejor rendimiento y es más rápido en comparación con MySQL, especialmente en contextos donde se manejan grandes cantidades de datos no estructurados o cuando se requiere flexibilidad en las operaciones de lectura y escritura de datos. [58]
Seguridad : MySQL es considerado más seguro debido a su estructura rígida, la cual ofrece mejor consistencia y fiabilidad en el manejo de los datos, así como su soporte para doble contraseña entre otros aspectos fundamentales para la seguridad. [58]

REFERENCES

[1] ORACLE, *"Database Administrator's Guide - Distributed Database Concepts"* [Online] : https://docs.oracle.com/en/database/oracle/oracle-database/19/admin/distributed-database-concepts.html

[2] M. Tamer Oszu, Patrick Valduriez, *"Princpiples of Distributed Database System"* [Online] : https://github.com/crafts-dev/E-Books/blob/master

[3] Saeed K., Rahimi S. Haug, Frank S. Haug. , *"Distributed Database Management System"* [Online] : https://github.com/crafts-dev/E-Books/blob/master

[4] Mohammad Altarade, *"The Definitive Guide to NoSQL Databases"* [Online] : https://www.toptal.com/database/the-definitive-guide-to-nosql-databases

[5] Dan Sullivan, *"NO-SQL For Mere Mortals"* [Online] : https://datubaze.wordpress.com/wp-content/uploads/2021/03/nosql-for-mere-mortals.pdf

[6] Raúl Herranz Gómez, *"BASES DE DATOS NOSQL: ARQUITECTURA Y EJEMPLOS DE APLICACIÓN"* [Online] : https://core.ac.uk/download/pdf/44310803.pdf

[7] ORACLE, *"Oracle® NoSQL Database SQL Beginner's Guide"* [Online] : https://docs.oracle.com/en/database/other-databases/nosql-database/23.3/sqlfornosql/sql-beginners-guide.pdf

[8] Eelco Plugge, Peter Membrey, Tim Hawkins, *"The Definitive Guide to MongoDB - The NoSQL Database for Cloud and Desktop Computing"* [Online] : https://pepa.holla.cz/wp-content/uploads/2016/11/The-Definitive-Guide-to-MongoDB.pdf

[9] AWS - Team, *" ¿Qué es una base de datos sin servidor? "* [Online] : https://aws.amazon.com/es/what-is/serverless-database/

[10] AWS - Team, *" Amazon Aurora Serverless "* [Online] : https://aws.amazon.com/es/rds/aurora/serverless/

[11] Jim Walker, Michelle Gienow, *" What is a serverless database? "* [Online] : https://www.cockroachlabs.com/blog/what-is-a-serverless-database/

[12] Datastax, *" What is a Serverless Database? "* [Online] : https://www.datastax.com/what-is/serverless-database

[13] Alex DeBrie, *" Choosing a Database for Serverless Applications "* [Online] : https://www.serverless.com/blog/choosing-a-database-with-serverless

[14] Charlie Custer, *" What is fault tolerance, and how to build fault-tolerant systems "* [Online] : https://www.cockroachlabs.com/blog/what-is-fault-tolerance/

[15] Imperva, *" Fault Tolerance "* [Online] : https://www.imperva.com/learn/availability/fault-tolerance/

[16] Kamil Braun, *" Making a Scalable and Fault-Tolerant Database System: Data Partitioning and Data Replication "* [Online] : https://www.scylladb.com/2020/10/20/making-a-scalable-and-fault-tolerant-database-system-partitioning-and-replication/

[17] Tutorialspoint, *"Checkpoints in DBMS "* [Online] : https://www.tutorialspoint.com/checkpoints-in-dbms

[18] Microsoft Build, *" Database checkpoints (SQL Server) "* [Online] : https://learn.microsoft.com/en-us/sql/relational-databases/logs/database-checkpoints-sql-server?view=sql-server-ver16

[19] Geeks For Geeks, *"Checkpoints in DBMS "* [Online] : https://www.geeksforgeeks.org/checkpoints-in-dbms/

[20] Flink, *" Checkpointing "* [Online] : https://nightlies.apache.org/flink/flink-docs-release-1.15/docs/dev/datastream/fault-tolerance/checkpointing/

[21] Javapedia.net, *" What is checkpointing in HDFS? "* [Online] : https://www.javapedia.net/Hadoop-basics/2214

[22] Microsoft Build, *" ¿Qué es Delta Lake? "* [Online] : https://learn.microsoft.com/es-es/azure/databricks/delta/

[23] The Linux Foundation Projects, *" Build Lakehouses with Delta Lake "* [Online] : https://delta.io/

[24] Project Pro, *" How to CREATE and LIST Delta Table in Databricks "* [Online] : https://www.projectpro.io/recipes/create-and-list-delta-table-databricks

[25] Quisitive, *" Querying Azure Databricks Delta Tables Directly Using Python "* [Online] : https://quisitive.com/querying-azure-databricks-delta-tables-directly-using-python/

[26] Databricks Documentation, *" Tutorial: Delta Lake "* [Online] : https://docs.databricks.com/en/delta/tutorial.html

[27] Databricks Documentation, *" Converting and ingesting data to Delta Lake "* [Online] : https://docs.databricks.com/en/delta/index.html

[28] The Linux Foundation Projects - Delta IO, *" Delta Lake "* [Online] : https://github.com/delta-io/delta

[29] Hewlett Packard, *" ¿Qué es Delta Lake? "* [Online] : https://www.hpe.com/mx/es/what-is/delta-lake.html

[30] PouchDB, *" PouchDB - Documentation "* [Online] : https://pouchdb.com/learn.html

[31] Josh Morony, *" Creating a Multiple User App with PouchDB & CouchDB "* [Online] : https://www.joshmorony.com/creating-a-multiple-user-app-with-pouchdb-couchdb/

[32] MongoDB, *" Introduction to MongoDB "* [Online] : https://www.mongodb.com/docs/manual/introduction/

[33] AWS - Team, *" Características de Amazon DynamoDB "* [Online] : https://aws.amazon.com/es/dynamodb/features/?pg=dynamodbt&sec=hs

[34] CouchDB, *" CouchDB - Introduction "* [Online] : https://docs.couchdb.org/en/stable/intro/index.html

[35] CockroachDB - Dev Team, *" CockroachDB "* [Online] : https://github.com/cockroachdb/cockroach

[36] Wikipedia, *" CockroachDB "* [Online] : https://en.wikipedia.org/wiki/CockroachDB

[37] Scifipete - Own work, *" The general structure of a relational database "* [Online] : https://en.wikipedia.org/wiki/Relational_database/media/File:RDBMS_structure.png

[38] AWS - Team, *" In The Works – Amazon Aurora Serverless "* [Online] : https://aws.amazon.com/es/blogs/aws/in-the-works-amazon-aurora-serverless/

[39] IBM Cloud, *" Riak "* [Online] : https://cloud.ibm.com/docs/database-tools?topic=database-tools-dbt-about-riak&locale=es

[40] Master of Big Data, *" RIAK: SISTEMAS NOSQL PARA TODOS LOS GUSTOS "* [Online] : https://master-bigdata.com/riak-sistemas-nosql-todos-los-gustos/

[41] Andrei Amador, *" Base de Datos No SQL - Riak "* [Online] : https://es.slideshare.net/andreicito1/bases-de-datos-nosql-riak

[42] MYSQL, *" MySQL NDB Cluster CGE "* [Online] : https://www.mysql.com/products/cluster/

[43] Wikipedia, *" MySQL Cluster "* [Online] : https://es.wikipedia.org/wiki/MySQL_Cluster

[44] Wikipedia, *" SQLite "* [Online] : https://en.wikipedia.org/wiki/SQLite

[45] Digital Guide Ionos, *" SQLite: la famosa biblioteca en detalle "* [Online] : https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/sqlite/

[46] Devopedia, *" SQLite "* [Online] : https://devopedia.org/sqlite

[47] NEO4J, *" La plataforma de datos grafos de Neo4j "* [Online] : https://neo4j.com/es/producto/

[48] Formadores IT, *" Base de datos orientada a grafos NEO4J "* [Online] : https://formadoresit.es/base-de-datos-orientada-a-grafos-neo4j/

[49] TodoBI, *" Introducción, análisis y uso de Neo4J "* [Online] : https://todobi.com/introduccion-analisis-y-uso-de-neo4j/

[50] *" "* [Online] : https://www.ionos.es/digitalguide/hosting/cuestiones-tecnicas/oracle-database/

[51] Digital Guide Ionos,*" Oracle Database: definición y funcionamiento "* [Online] :

[52] Wikipedia, *" Oracle Database "* [Online] :https://en.wikipedia.org/wiki/Oracle_Database

[53] Pure Storage, *" ¿Qué es Delta Lake? Una descripción general "* [Online] : https://www.purestorage.com/la/knowledge/what-is-delta-lake.html#: :text=Delta%20Lake%20es%20una%20capa,y %20aprendizaje%20automático%20(ML).

[54] Andrew S. Tanenbaum, Maarten Van Steen ,*" Distributed Systems: Principles and Paradigms "* [Online] : https://github.com/seawish/books/blob/master/books/

[55] IBM, *" ¿Qué es el teorema CAP? "* [Online] : https://www.ibm.com/mx-es/topics/cap-theorem

[56] Israel Koren , C. Mani Krishna. *" Fault-Tolerant Systems "* [Online] : https://github.com/sap9433/IIT-CS550-Advance-OS-Assignments/blob/master/Books/

[57] PostgreSQL - Team *" PostgreSQL 16.2 Documentation "* [Online] : https://www.postgresql.org/files/documentation/pdf/16/postgresql-16-A4.pdf

[58] KInsta, Web Development Tools, *" MongoDB vs MySQL: Which Is the Better Database Management System? "* [Online] : https://kinsta.com/blog/mongodb-vs-mysql/

[59] Eric Redmond, John Daily, *" Little Riak Book "* [Online] : https://github.com/basho-labs/littleriakbook/blob/master/rendered/riaklil-print-en.pdf